OpenClaw Hackathon by Samsung Prism 2026 - Daily Utility Track

CONTEXTA

CONNECT • UNDERSTAND • TRANSFORM

Personal AI Operating System for Smartphones

Adapts phone behaviour from context — silently, on-device, without user input.

TEAM BETA ONEPIECE

M.S.RAMAIAH INSTITUTE OF TECHNOLOGY

Akshay A - 1MS24IS013

Team Lead · Frontend

Aaditya V - 1MS24IS001

Backend

Tejas M - 1MS24CI134

UI/UX

H M Pranav - 1MS24IS047

Database

PROBLEM STATEMENT

Smartphones are reactive by design

Static defaults don't fit dynamic lives

Ringtones, brightness, and app layouts stay fixed regardless of where you are or what you're doing.

Context switching is manual and forgettable

Users must remember to enable Do Not Disturb before every meeting, adjust settings at every location change.

~40 setting adjustments per user per day

Each one is low-stakes individually. Collectively, they fragment focus and cost minutes throughout the day.

The phone has all the data it needs to be proactive. It just doesn't use it.

GAP ANALYSIS

Why existing tools fall short

Tool	What It Does	Critical Gap
Do Not Disturb	Silences on a fixed schedule	No context awareness at all
IFTTT / Tasker	Rule-based trigger → action	Breaks on edge cases; requires setup
Google Assistant	Responds to voice commands	Passive — you still initiate
Android Focus Modes	Manual profile switching	User must remember to switch
Pixel Call Screening	Single-purpose AI feature	Narrow; no general context model
Contexta ★	**Infers context; acts proactively**	**Gap addressed ✓**

SOLUTION OVERVIEW

Contexta—Context-Aware Phone Intelligence

Contexta reads your environment, learns your habits, and adjusts your phone — before you think to do it. Built for students and professionals with structured daily routines. High-value scenarios: meetings, classes, commuting.

- **Reads signals** — location, calendar, time, recent app usage
- **Classifies context** — meeting, commute, sleep, leisure, focus
- **Dispatches actions** — silent mode, brightness, DND, app priority
- **Learns corrections** — user overrides feed back into the local model
- **Stays on-device** — no cloud inference; privacy-preserving by design

SCOPE BOUNDARY

Silent / DND automation

Brightness control

App surface ranking

ML pattern engine

Cross-device sync

OEM integration

CORE CONCEPT

Observe → Decide → Act (continuous loop)

OBSERVE

- GPS + geofence events
- Calendar / event NLP
- Clock + day-of-week
- App open/close deltas
- Notification types

DECIDE

- Context classifier (TFLite)
- Rule overlay for edge cases
- Confidence threshold check
- Action priority scoring
- Conflict resolution logic

ACT + LEARN

- Android Settings API calls
- Notification policy changes
- App surface adjustments
- User override logged
- Model update (local)

All three stages fire on a 30-second WorkManager polling cycle. No user input required at any stage.

SYSTEM FLOW

Step-by-step decision pipeline

1

Signal Ingestion

GPS + Calendar + Clock polled every 30s by WorkManager background job

2

Feature Extraction

Signals → feature vector: {location_class, event_type, hour_bin, day_type}

3

Context Classification

TFLite 3-layer MLP (~50KB) classifies into: Meeting / Commute / Sleep / Focus / Leisure

4

Rule Overlay

Hard rules check: emergency contacts, manual override flag, conflict detection

5

Action Dispatch

Spring Boot receives context label → calls Android Settings API via companion app

6

Feedback Capture

User overrides logged to PostgreSQL → batch retraining scheduled nightly

KEY FEATURES

What Contexta does — grouped by capability

CONTEXT SENSING

- Location geofencing (home / work / other)
- Calendar NLP: title keyword extraction
- Time + weekday pattern detection

ADAPTIVE ACTIONS

- Silent / DND auto-toggle
- Brightness + screen timeout adjustment
- Notification channel suppression

ON-DEVICE AI

- TFLite classifier: 50KB, <5ms latency
- Rule + ML hybrid for reliability
- Nightly incremental local retraining

USER TRUST LAYER

- One-tap override at any time
- Full audit log of every auto-action
- Per-context sensitivity slider

DIFFERENTIATION

Rule-based automation vs. learned context

RULE-BASED SYSTEMS	CONTEXTA (HYBRID AI)
✕ Explicit rules for every case	✓ Classifies context; generalises to unseen situations
✕ Breaks silently on edge cases	✓ Falls back to safe defaults; logs anomalies
✕ Static — requires manual rule updates	✓ Adapts incrementally from user corrections
✕ Binary if/then triggers	✓ Probabilistic scores with confidence thresholds
✕ No memory across sessions	✓ Behavioural model persists and improves over time

Note: AI here means a lightweight on-device TFLite classifier — not a large language model.

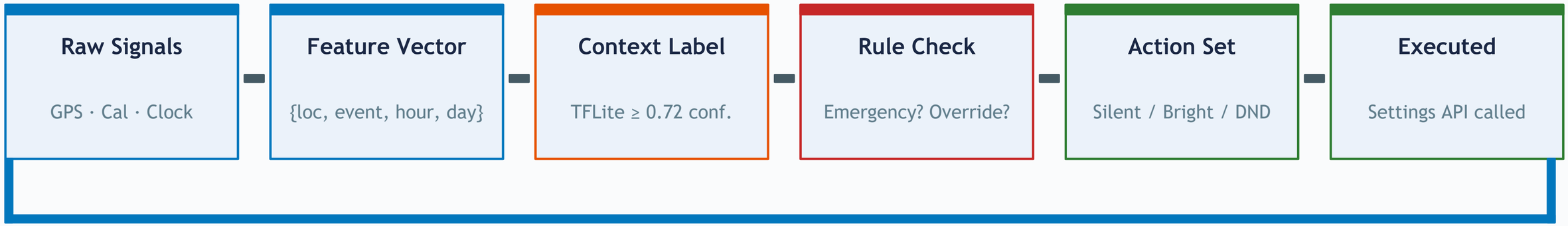
Component Architecture Overview

UI LAYER	React Native App	Override Controls	Settings Dashboard	Audit Log View
ORCHESTRATION	Spring Boot API	Action Dispatcher	Event Bus	REST Endpoints
INTELLIGENCE	TFLite Classifier	Rule Engine	Feature Extractor	Feedback Processor
DATA / PERSISTENCE	PostgreSQL 16	SQLite (on-device)	Redis 7	Event Log Store
SENSOR / PLATFORM	GPS / Location API	Calendar Provider	WorkManager	Notification Mgr

TFLite runs fully on-device. Backend receives only context labels — never raw location or calendar data. System functions completely without backend dependency in offline mode.

DATA FLOW

Decision pipeline — input to action



User Override → Feedback Loop → Logged to DB → Nightly Retraining

Confidence threshold logic:

- ≥ 0.80 confidence → action dispatched automatically
- 0.55 – 0.79 → action dispatched with silent review log
- < 0.55 → fallback to last known state; no action taken

REAL-LIFE SCENARIO

A Tuesday — Rohan's day, zero manual changes

7:48 AM Alarm dismissed · GPS at home

Morning mode: max brightness · news widget surfaced

8:22 AM GPS: moving, speed > 20 km/h

Maps launched proactively · podcast prioritised

9:00 AM Calendar: 'Sprint Standup' detected

Silent mode on · Slack muted · DND activated

9:48 AM Calendar event ended

Notifications restored · email sorted by priority

1:05 PM GPS: lunch location pattern recognised

Screen timeout extended · media apps surfaced

6:40 PM GPS: home geofence entered

Work profile paused · personal mode activated

6 context transitions. 0 manual setting changes. Phone adapted throughout the day.

DEMO WALKTHROUGH

From Context to Action — Live Flow

01

Dashboard overview

Live context panel: location class, next calendar event, confidence score.

↳ *Open app → Context tab*

02

Live signal feed

Signal inspector: GPS updates, calendar pull, hour-bin — all live.

↳ *Tap 'Signal Monitor'*

03

Meeting simulation

Inject mock event 'Client Call'. Phone silences, Slack muted, DND on — no input.

↳ *'Inject Event' debug btn*

04

Confidence display

Classifier returns 'Meeting' at 0.91. Score breakdown per class shown.

↳ *Classifier Debug panel*

05

Override + audit log

Restore sound manually. Audit log: action, timestamp, confidence, override flag.

↳ *Override → Audit tab*

06

End state summary

6 auto-actions · 1 override · 0 manual adjustments. Override feeds correction back.

↳ *Summary card on home*

IMPLEMENTATION STATUS

What is built · partial · planned

Context classifier (TFLite)

3-layer MLP on 2,000 synthetic + 500 real behavioural samples

Calendar NLP keyword parser

Regex + keyword list for meeting/event detection (no LLM)

Spring Boot REST API

CRUD: context_log, action_log, override_capture endpoints

Brightness adaptive control

Logic written; permission prompt flow incomplete on physical device

Incremental retraining

Batch script runs; not yet wired into WorkManager schedule

Per-user model personalisation

Federated averaging approach identified; not yet implemented

Location geofencing

Android Fused Location API — home / work / other 3-class

Silent mode + DND dispatch

Tested on Pixel 7 emulator; Settings API integration confirmed

PostgreSQL schema + ORM

Hibernate entities: context_events, action_log, user_corrections

App surface re-ranking

Algorithm designed; Android Launcher API integration 70% done

Cross-device sync

Architecture sketched; needs Google Identity + device link layer

OEM / system-level hooks

Requires signed OEM agreement; out of hackathon scope

PROTOTYPE EVIDENCE

What we have running on emulator

contexta/ classifier_output.log

```
09:00:03.412  [WorkManager-1]
INFO  SignalCollector: GPS fix acquired
INFO  location_class=WORK (geofence hit)
INFO  CalendarParser: event=Sprint Standup
INFO  event_type=MEETING  hour_bin=9
09:00:03.489  [TFLiteClassifier]
INFER  context=MEETING  conf=0.91
scores: {MTG:0.91 CMT:0.05 SLP:0.02
         FOC:0.01 LSR:0.01}
09:00:03.501  [ActionDispatcher]
ACTION  RINGER_MODE -> SILENT   [OK]
ACTION  DND_POLICY -> PRIORITY [OK]
09:00:03.512  [AuditLogger]
DB WRITE  action_log id=1482  [201 OK]
override_flag=false  latency=89ms
```

✓ Silent mode verified on emulator
Pixel 7 API 33 emulator. RINGER_MODE_SILENT set via AudioManager. Confirmed with getSystemService() poll before/after event injection.

✓ Classifier output: Meeting (0.91)
TFLite MLP. Input: {WORK, MEETING, 9, WEEKDAY}. Exceeds 0.80 threshold; action auto-dispatched. Scored all 5 classes.

✓ Audit log entry written to PostgreSQL
Spring Boot REST returned HTTP 201. Row: context=MEETING, action=SILENT, conf=0.91, override=false, latency=89ms.

✓ UI — 3 screens in React Native
Context tab (live score card), Signal Monitor (raw feed), Audit tab (timestamped action list). Override toggle working.

The real challenges — and how we address them

Battery & Background Restrictions

Problem: Android Doze mode kills background services after ~10 min inactivity.

Fix: WorkManager with periodic tasks (min 15-min interval). Accepting some latency tradeoff.

Android Permission Wall

Problem: WRITE_SETTINGS, ACCESS_BACKGROUND_LOCATION, PACKAGE_USAGE_STATS all require explicit grants; some need OEM permissions.

Fix: Onboarding requests permissions one-at-a-time with justification. Degraded mode if denied.

Privacy & Data Minimisation

Problem: Location + calendar data is sensitive. Sending raw signals to a server is a hard no for most users.

Fix: All inference on-device via TFLite. Backend receives only context labels — never raw location or calendar data.

Model Accuracy vs. Action Cost

Problem: A misclassification that silences calls during an emergency is worse than doing nothing.

Fix: 0.55 confidence floor. Emergency contacts always bypass DND. Overrides log negative reward signal.

TECH STACK

Realistic, production-grade tool choices

MOBILE / FRONTEND	**React Native 0.73** Cross-platform UI	**Expo SDK 50** Dev + OTA updates	**Redux Toolkit** State management
BACKEND SERVICES	**Java 17 + Spring Boot 3** REST API server	**Spring Security 6** JWT auth	**WebSocket (STOMP)** Real-time push
AI / ML	**TensorFlow Lite 2.14** On-device inference	**Scikit-learn** Offline training	**Custom Pipeline** Signal normalisation
DATA LAYER	**PostgreSQL 16** Event + action log	**SQLite (on-device)** Local cache/offline	**Redis 7** Session + API cache
INFRA / TOOLING	**Android WorkManager** BG scheduling	**Fused Location API** GPS / geofence	**JUnit 5 + Mockito** Test coverage

Near-term value and honest next steps

~40	3.6B	<5ms	Zero
fewer taps/day (estimated)	Android devices addressable	classifier latency	raw data leaves device

Short term (1 month)

Physical device testing on 3+ Android models · Brightness + app ranking fully wired · Delta fine-tuning

Medium term (3 months)

Open beta: 50 real users · Privacy audit vs GDPR / India DPDP · Android 14 restricted settings compat

Long term

OEM partnership for system integration · Cross-device sync · Accessibility adaptive mode

THE CASE IN ONE LINE

Your phone already has everything it needs to help you.

Contexta makes it act on it — on-device, privately, without asking.